

# TU Hospital Management System

## A Django 5 Hospital Information System for Nepal

Complete System & Software Design Documentation

Architecture • ER & Class Diagrams • Use-Case, Sequence & State Diagrams •  
Module-by-Module Reference • Data Dictionary

---

|                        |                                                                                                                                                          |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project</b>         | TU Hospital Management System (Nepal)                                                                                                                    |
| <b>Build stage</b>     | Checkpoint 5 of a multi-part build (all 5 checkpoints included)                                                                                          |
| <b>Framework</b>       | Django 5 (Python), Django ORM, server-rendered templates, Bootstrap-style CSS                                                                            |
| <b>Modules</b>         | Website, Accounts, Departments, Doctors, Patients, Appointments, Consultations, Laboratory, Radiology, Admissions, Billing, Pharmacy, Insurance, Reports |
| <b>Staff roles</b>     | Super Admin, Registration Counter, Cash Counter, Doctor, Pharmacy, Laboratory, Radiology Counter, Insurance Counter                                      |
| <b>Local specifics</b> | Full Nepal address system (7 Provinces / 77 Districts), eSewa payment gateway, Patient IDs like TUH-2026-000001                                          |
| <b>Document type</b>   | Auto-generated technical documentation                                                                                                                   |

This document was generated directly from the project's source code (models, views, forms, and URL configuration) inside the submitted ZIP archive, so every field, workflow step, role, and status value shown here matches the actual implementation.

## Table of Contents

- 1. Introduction & Project Overview
- 2. System Architecture
- 3. Roles & Access Control
- 4. Module-by-Module Functional Description
- 5. Entity-Relationship Diagrams
- 6. Use-Case Diagram
- 7. Key Workflows (Sequence & State Diagrams)
- 8. Data Dictionary
- 9. Identifier & Numbering Schemes
- 10. Auditing, Security & Data Integrity
- 11. Technology Stack & Deployment
- 12. Build Status & Roadmap

# 1. Introduction & Project Overview

TU Hospital Management System is a Django 5 web application built module-by-module for hospitals operating in Nepal. It combines a public marketing/informational website with a full back-office hospital information system: patient registration, online appointment booking with eSewa payment, doctor consultations, laboratory and radiology workflows, in-patient admission & discharge, cash billing, a pharmacy with inventory control, and an insurance claims desk — all tied together by role-based staff accounts and a system-wide, append-only audit log.

The system is built specifically for the Nepali context: patient and appointment forms use the real Nepal Province/District hierarchy (7 Provinces, all 77 Districts, seeded and searchable), online payments go through the eSewa gateway, and every permanent record (patient, visit, bill, claim, admission, sale) gets a human-readable sequential ID such as TUH-2026-000001.

## 1.1 What this document covers

- The overall architecture and how the 14 Django apps relate to each other.
- Every staff role and exactly what each role can access.
- A functional walkthrough of every module in the system.
- Entity-Relationship diagrams covering the full data model.
- A system-wide use-case diagram.
- Sequence and state diagrams for the two most important workflows: the OPD patient journey and in-patient admission/discharge.
- A complete data dictionary, ID-numbering reference, and security/audit notes.

# 2. System Architecture

The application follows Django's standard MVT (Model-View-Template) pattern. Each business area is its own Django app with its own models, views, forms and URLs, all mounted under one project (`config/`). Media files (QR codes, uploaded photos, lab/radiology reports) are served from `media/`, and online appointment payments are redirected to the eSewa gateway and back.

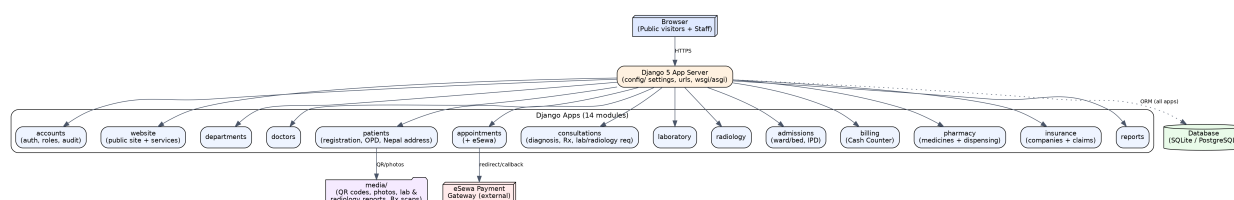


Figure 2.1 — Component architecture: all 14 Django apps, the database, media storage, and the external eSewa gateway.

## 2.1 Project folder structure

```

config/ (settings, urls, wsgi/asgi) • accounts/ • website/ • departments/ • doctors/ •
patients/ • appointments/ • consultations/ • laboratory/ • radiology/ • admissions/ •
billing/ • pharmacy/ • insurance/ • reports/ • templates/ • static/ • media/
  
```

### 3. Roles & Access Control

Every staff member logs in with a role stored on the custom User model. Login redirects each role straight to its own dashboard, and every view that isn't public is protected by a role-checking decorator (e.g. `@doctor_required`, `@super_admin_required`). A Django superuser is always treated as Super Admin regardless of the stored role field.

| Role                 | Login redirects to                            | Primary responsibilities                                                                  |
|----------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------|
| Super Admin          | /accounts/dashboard/super-admin/              | Master data (departments, doctors, services, medicines, insurers), staff/user management  |
| Registration Counter | /patients/dashboard/                          | Patient registration, OPD visits, QR/ticket printing, patient search, 24-hour edit window |
| Cash Counter         | /billing/                                     | Patient lookup, multi-service billing, Cash/eSewa/Insurance payment, receipts, totals     |
| Doctor               | /doctors/dashboard/ (queue at /consultation/) | Consultation queue, diagnosis, prescriptions, lab & radiology requests, patient history   |
| Pharmacy             | /pharmacy/                                    | Medicine dispensing, stock validation, pharmacy billing, today's sales                    |
| Laboratory           | /laboratory/                                  | Lab request queue, status updates, result upload, printable reports                       |
| Radiology Counter    | /radiology/                                   | Imaging request queue, status updates, report/image upload                                |
| Insurance Counter    | /insurance/                                   | Patient insurance profile, claim submission & review, claims reporting                    |

## 4. Module-by-Module Functional Description

### 4.1 Public Website (website app)

Home, About, Departments, Doctors, Diseases, Contact, and Gallery pages for public visitors, backed by Super-Admin-managed content: testimonials, gallery images, disease information, and the master Hospital Services catalogue used for billing throughout the system.

### 4.2 Accounts (auth, roles, audit)

A custom User model extends Django's AbstractUser with a `role` field, plus Super-Admin screens to create/edit/activate any staff account. Every sensitive action across every module writes to the shared, append-only `AuditLog`.

### 4.3 Departments & Doctors

Super Admin maintains hospital Departments (with consultation fee, description, photo) and Doctors (qualification, specialization, schedule, consultation fee, photo). Doctors optionally link to their own login account for the Doctor role.

### 4.4 Patients (registration & OPD)

Patient registration captures identity, the seeded Nepal Province/District (with free-text Municipality/Ward/Tole), optional insurance details, and generates a permanent Patient ID, QR code, and printable OPD Ticket/Patient Card. A 24-hour free-edit window is enforced; edits after that window create a new, separately charged Visit rather than silently altering history.

### 4.5 Appointments (public booking + eSewa)

Public visitors book an appointment by department/doctor/date, pay online through eSewa's sandbox or live gateway, and receive an appointment number, QR code, and a printable/PDF receipt. Payment status is tracked (Pending / Paid / Failed) with the eSewa reference ID.

### 4.6 Consultations (Doctor module)

Each doctor works through a queue built from their own OPD visits. A consultation records diagnosis and clinical notes, and lets the doctor build a dynamic prescription (medicine, dosage, frequency, duration, instructions) and request laboratory tests or radiology imaging in the same screen, with full per-patient consultation history for continuity of care.

### 4.7 Laboratory

Runs entirely on the `LabTestRequest` data the Doctor module writes — no separate table is needed. Provides a dashboard (pending / in-progress / completed-today / urgent), a filterable queue, patient search, and a status-update screen that attaches result notes and an uploaded result file, plus a printable lab report.

### 4.8 Radiology

Mirrors the Laboratory module for imaging: X-Ray, CT, MRI, ECG, Echo, Ultrasound, or a custom service, each with its own permanent RAD-2026-000001 number, urgency, status workflow (Requested → Scheduled → In Progress → Completed), uploaded images, and a report with findings/impression.

## 4.9 Admissions (in-patient / Ward & Bed)

Tracks physical Wards (General, ICU, Private, Emergency, Maternity, Pediatric) and their Beds, and an Admission record from admit through discharge: reason for admission, diagnosis, admitting doctor, assigned ward/bed, and on discharge a condition (Recovered / Improved / Referred / DAMA / Expired), discharge summary, and follow-up instructions. Discharging an admission automatically frees its bed.

## 4.10 Billing (Cash Counter)

Patient lookup by ID, QR, phone, or name; a patient detail panel showing insurance info and QR; multi-service billing with automatic totals; Cash / eSewa / Insurance payment methods; printable A4/thermal receipts; a separately audited reprint flow that never touches the original bill; and a personal "Today's Collections" report.

## 4.11 Pharmacy + Inventory

Super-Admin-only Medicine catalogue (code, generic/brand name, category, strength, unit, purchase/selling price, stock, minimum-stock alert level, batch, expiry, manufacturer) with low-stock, out-of-stock, near-expiry, and expired badges. A Stock Adjustment tool records every purchase, correction, write-off, or patient return against an audit-safe history that never edits past records. The Pharmacy role dispenses against digital prescriptions (linked to a consultation), manual/paper prescriptions (with an optional scanned copy), or walk-in sales, with live stock validation and automatic price calculation; stock is decremented atomically at sale time and can never go negative.

## 4.12 Insurance (Companies + Claims)

Super Admin maintains the insurer directory. The Insurance Counter searches a patient, views their policy and full claim history, submits a new claim (amount claimed, co-payment, cashless flag, related receipt), and reviews pending claims to Approve, Reject, or Settle them — each transition logged to the audit trail. Insurance can also be selected directly as a Bill/PharmacySale payment method, independent of filing a reimbursement claim.

## 4.13 Reports

A consolidated reporting area (income, department, doctor, counter, pharmacy, laboratory, registrations, and audit-trail views) that reads across the other modules' data for management visibility, alongside each module's own in-module report screen (e.g. Insurance Reports, Today's Collections, Today's Sales).

## 5. Entity-Relationship Diagrams

The data model is presented in two diagrams for readability: the clinical/operational core (patients, visits, doctors, consultations, lab/radiology requests, admissions, appointments), and the financial/inventory side (billing, pharmacy sales, medicines, insurance).

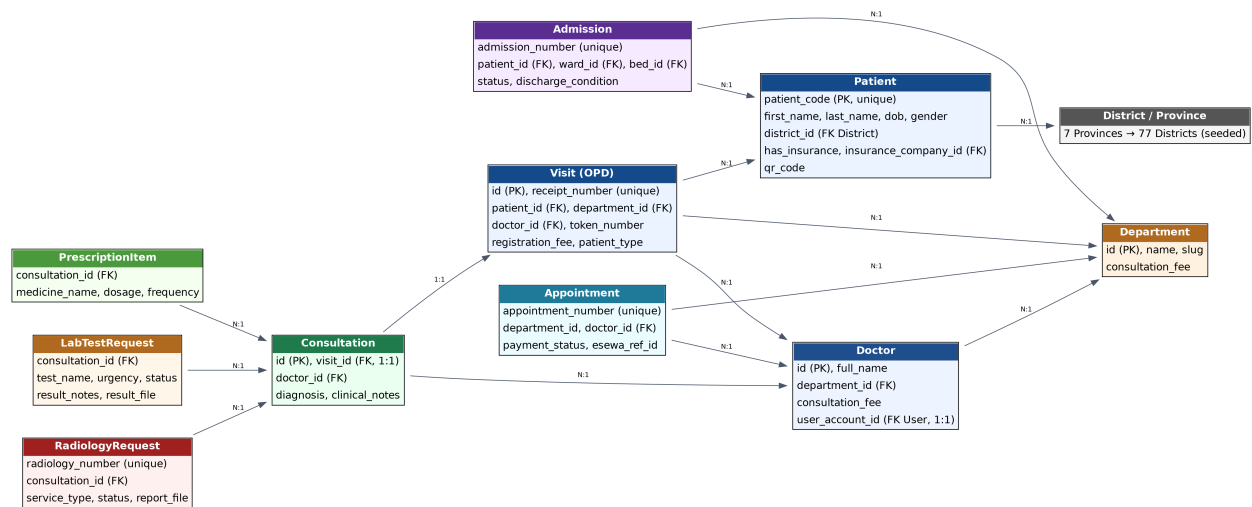


Figure 5.1 — Core clinical/operational entities: Patient, Visit, Doctor, Department, Consultation, Prescription, Lab & Radiology requests, Admission, Appointment.

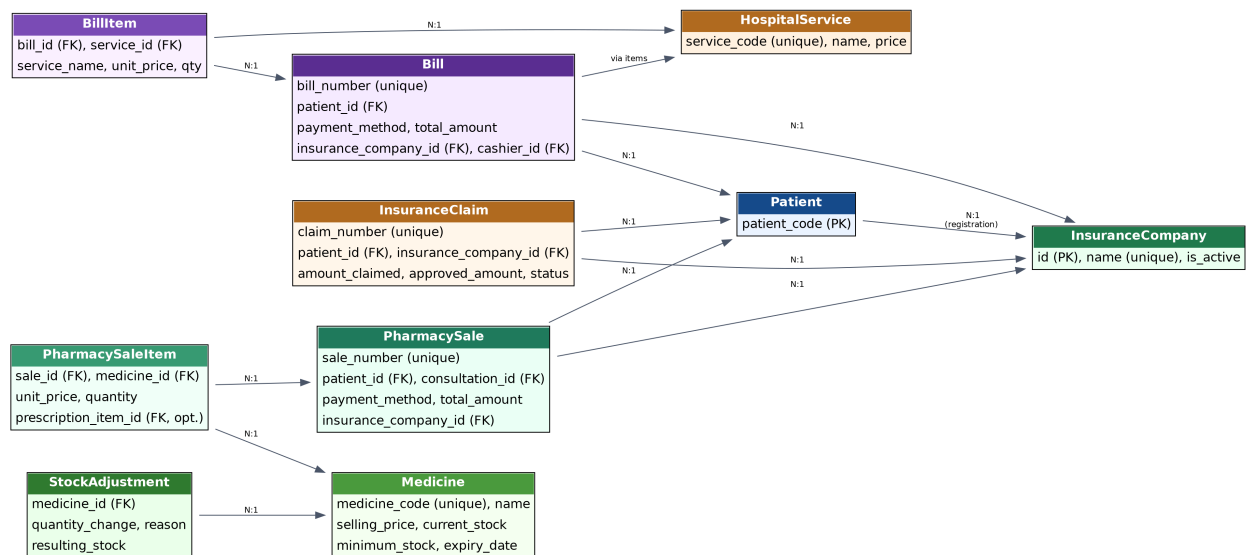


Figure 5.2 — Financial & inventory entities: *Bill*, *BillItem*, *HospitalService*, *Medicine*, *StockAdjustment*, *PharmacySale*, *PharmacySaleItem*, *InsuranceCompany*, *InsuranceClaim*.

All permanent transactional records (*Patient*, *Visit*, *Bill*, *PharmacySale*, *InsuranceClaim*, *Admission*, *Appointment*, *RadiologyRequest*) follow the same pattern: a unique, human-readable number generated inside an atomic, row-locked transaction, so numbers stay gapless and collision-free even under concurrent counter activity.



## 6. Use-Case Diagram

The diagram below summarizes every actor in the system and the main use cases each one performs, including the automatic audit-logging that `<<include>>` applies to sensitive actions, and how a consultation `<<extend>>`s into lab, radiology, and pharmacy work.

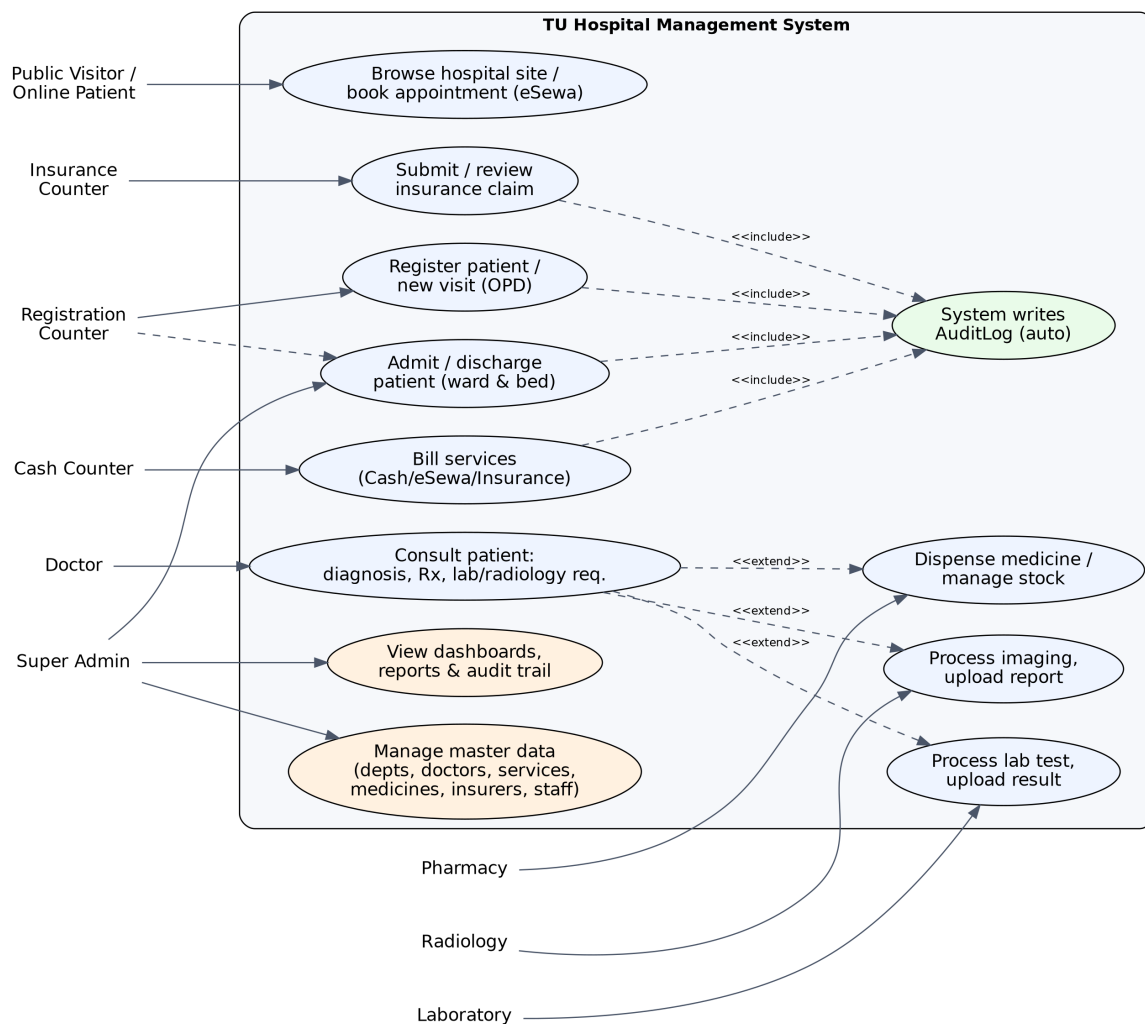


Figure 6.1 — System-wide use-case diagram, all staff roles and the public visitor.

## 7. Key Workflows

### 7.1 The OPD patient journey (sequence diagram)

This is the most common end-to-end path through the system: a walk-in patient moving from registration through consultation, optional lab/radiology work, pharmacy, and finally billing.

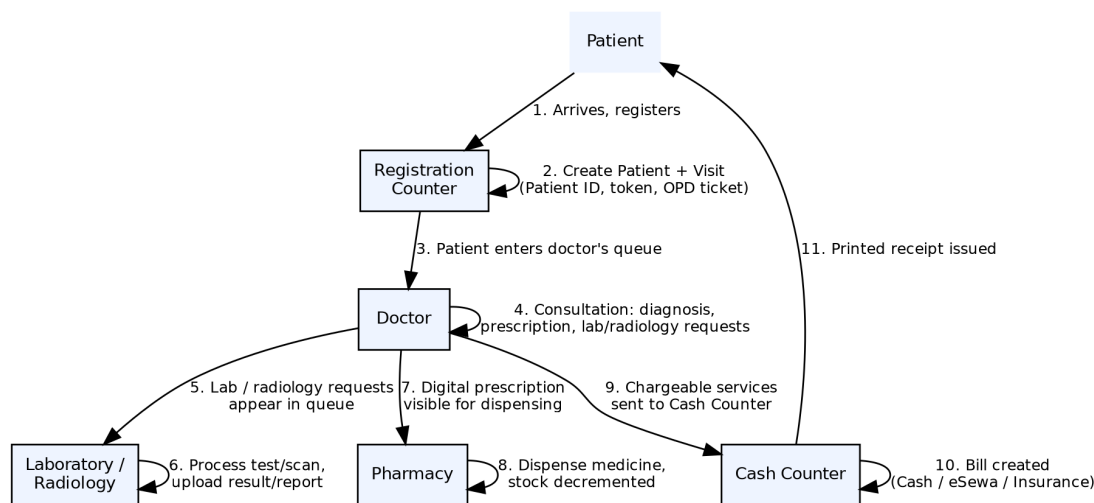


Figure 7.1 — Sequence diagram: Registration → Consultation → Laboratory/Radiology → Pharmacy → Cash Counter.

### 7.2 In-patient admission & discharge (state diagram)

An Admission moves through a small, well-defined state machine from the moment a ward/bed is assigned to final discharge.

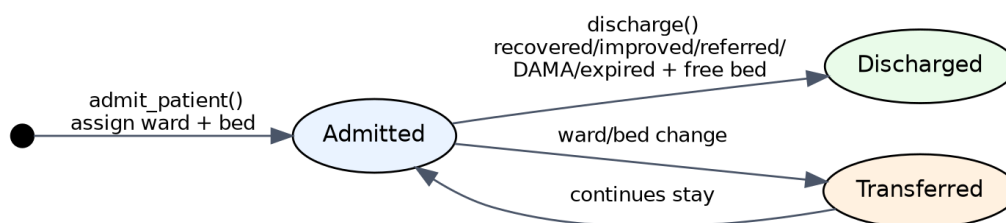


Figure 7.2 — Admission status state diagram.

### 7.3 Online appointment payment (eSewa)

- Visitor fills the public booking form (department, doctor, preferred date/time, patient details).
- An Appointment is created with a unique `transaction_uuid` and payment status Pending.
- The visitor is redirected to eSewa's sandbox/live payment page to pay the registration fee.
- On successful payment, eSewa redirects back; the system verifies and calls `mark_paid()`, which generates the permanent appointment number, sets status to Paid, records the eSewa reference ID, and generates a QR code.
- A printable/PDF receipt is issued; a failed payment routes to a Payment Failed page.

## 8. Data Dictionary

Field-level reference for the primary model in each module. Full audit fields (created\_at/updated\_at, created\_by, etc.) are included; helper/computed properties are omitted for brevity.

### 8.1 Patient (patients app)

| Field                                                             | Type                       | Notes                                                  |
|-------------------------------------------------------------------|----------------------------|--------------------------------------------------------|
| patient_code                                                      | CharField, unique, indexed | Auto-generated: TUH-2026-000001                        |
| first_name / last_name                                            | CharField                  | Required                                               |
| gender / date_of_birth                                            | Choice / Date              | Required                                               |
| phone_number                                                      | CharField                  | Required                                               |
| district                                                          | ForeignKey → District      | PROTECT; part of seeded 7-Province/77-District dataset |
| municipality / ward_number / local_address                        | CharField / Char           | Free-text (753 local levels in Nepal)                  |
| citizenship_number, email, blood_group, occupation, guardian_name | CharField                  | Optional                                               |
| agency_contact                                                    | CharField                  | Optional                                               |
| has_insurance + 5 insurance_* fields                              | Bool + various             | Captured at registration; linked to InsuranceCompany   |
| qr_code                                                           | ImageField                 | Auto-generated from patient_code on save               |
| created_by / created_at / updated_at                              | FK User / DateTime         | Audit fields                                           |

### 8.2 Visit (OPD registration, patients app)

| Field                         | Type              | Notes                                                      |
|-------------------------------|-------------------|------------------------------------------------------------|
| receipt_number                | CharField, unique | REC-20260712-00001                                         |
| patient / department / doctor | ForeignKeys       | doctor is optional at registration time                    |
| patient_type                  | Choice            | New / Old — determines registration_fee                    |
| token_number                  | PositiveInteger   | Per department, per day                                    |
| is_correction_of              | Self-FK, nullable | Links a post-24-hour re-registration to the original visit |

### 8.3 Consultation, PrescriptionItem, LabTestRequest (consultations app)

| Model            | Key fields                                                                  | Notes                                       |
|------------------|-----------------------------------------------------------------------------|---------------------------------------------|
| Consultation     | visit (1:1), doctor, diagnosis, clinical_notes, follow_up_date              | One-to-one Visit                            |
| PrescriptionItem | medicine_name, dosage, frequency, duration, instructions                    | File-text medicine name                     |
| LabTestRequest   | test_name, urgency, status, result_notes, result_file                       | Status: Requested / In Progress / Completed |
| RadiologyRequest | radiology_number, service_type, urgency, status, report_text, report_number | Own codes, report number, RAD-2026-000001   |

## 8.4 Admission / Ward / Bed (admissions app)

| Field                                      | Type                  | Notes                                                        |
|--------------------------------------------|-----------------------|--------------------------------------------------------------|
| admission_number                           | CharField, unique     | ADM-2026-000001                                              |
| patient / department / ward / bed          | ForeignKeys           | bed is nullable (ward-level admission before bed assignment) |
| admitting_doctor                           | FK → Doctor, nullable |                                                              |
| status                                     | Choice                | Currently Admitted / Discharged / Transferred                |
| discharge_condition                        | Choice                | Recovered / Improved / Referred / DAMA / Expired             |
| discharge_summary / follow_up_instructions | Text                  | Filled at discharge                                          |
| length_of_stay_days                        | computed property     | discharge_date (or now) minus admission_date                 |

## 8.5 Bill / BillItem (billing app)

| Field                                                 | Type                  | Notes                                                                     |
|-------------------------------------------------------|-----------------------|---------------------------------------------------------------------------|
| bill_number                                           | CharField, unique     | BILL-20260712-00001                                                       |
| patient / cashier / insurance_company                 | ForeignKeys           | insurance_company set when payment_method=insurance                       |
| payment_method                                        | Choice                | Cash / eSewa / Insurance                                                  |
| total_amount / status                                 | Decimal / Choice      | Status: Paid / Refunded                                                   |
| BillItem: service, service_name, unit_price, quantity | ForeignKey to service | Price/name snapshotted at billing time for audit-safe historical receipts |

## 8.6 Medicine / StockAdjustment / PharmacySale (pharmacy app)

| Model            | Key fields                                                                            | Notes                                                                          |
|------------------|---------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Medicine         | medicine_code, name, category, strength, unit, purchase_price                         | selling price, stock, use-by date, expiry date, near-expiry date, expired date |
| StockAdjustment  | quantity_change, reason, resulting_stock, adjusted_by                                 | Reason: Purchase / Correction / Expired-Write-off / Patient Return             |
| PharmacySale     | sale_number, patient, consultation, prescription_source, payment_method, total_amount | prescription, total amount, Digital / Manual / WhatsApp                        |
| PharmacySaleItem | medicine, medicine_name, unit_price, quantity, prescription_item                      | Snapshoted name/price; optional link back to the doctor's prescription         |

## 8.7 InsuranceCompany / InsuranceClaim (insurance app)

| Field                                                                                 | Type              | Notes                                   |
|---------------------------------------------------------------------------------------|-------------------|-----------------------------------------|
| InsuranceCompany: name, contact_person, phone_number, email, address, insurance_image | Person            | Super Admin managed                     |
| InsuranceClaim.claim_number                                                           | CharField, unique | CLAIM-2026-000001                       |
| patient / insurance_company / submitted_by                                            | ForeignKeys       | Reviewed by                             |
| amount_claimed / approved_amount / rejected_amount / co_payment_amount                | Decimal(10, 2)    |                                         |
| is_cashless                                                                           | Boolean           |                                         |
| status                                                                                | Choice            | Pending / Approved / Rejected / Settled |

## 8.8 Appointment (appointments app)

| Field                                     | Type                     | Notes                                         |
|-------------------------------------------|--------------------------|-----------------------------------------------|
| appointment_number                        | CharField, unique        | APT-2026-000001, generated on payment success |
| department / doctor / district            | ForeignKeys              |                                               |
| patient_type / registration_fee           | Choice / Decimal         |                                               |
| transaction_uuid                          | UUID, unique             | Generated immediately, before payment         |
| payment_status / esewa_ref_id / paid_date | Choice / Char / DateTime | Pending / Paid / Failed                       |

## 9. Identifier & Numbering Schemes

Every module that creates a permanent, citable record uses the same safe pattern: generate the number inside `transaction.atomic()` using `select_for_update()` on the highest existing number for the current prefix, so concurrent submissions from different counters can never produce a duplicate.

| Record            | Format                            | Example             |
|-------------------|-----------------------------------|---------------------|
| Patient ID        | &lt;HOSPITAL_CODE&gt;-YYYY-000001 | TUH-2026-000001     |
| OPD Visit receipt | REC-YYYYMMDD-00001                | REC-20260712-00001  |
| Appointment       | APT-YYYY-000001                   | APT-2026-000001     |
| Cash Counter bill | BILL-YYYYMMDD-00001               | BILL-20260712-00001 |
| Pharmacy sale     | PH-YYYYMMDD-00001                 | PH-20260712-00001   |
| Insurance claim   | CLAIM-YYYY-000001                 | CLAIM-2026-000001   |
| Admission         | ADM-YYYY-000001                   | ADM-2026-000001     |
| Radiology request | RAD-YYYY-000001                   | RAD-2026-000001     |

## 10. Auditing, Security & Data Integrity

- A single, shared `AuditLog` model (accounts app) records login, logout, patient registration/update, new-visit corrections, payments, reprints, refunds, appointment bookings, admissions/discharges, and insurance claim submission/review.
- The audit log is append-only: no module edits or deletes an existing audit row; corrections create a new record instead.
- Financially sensitive lines (`BillItem`, `PharmacySaleItem`) snapshot the service/medicine name and price at transaction time, so a later price change by Super Admin never rewrites historical receipts.
- Reprinting a bill or receipt is a separately audited action that never modifies the original record.
- Role-based decorators (e.g. `@super_admin_required`, `@insurance_required`) gate every non-public view; a Django superuser is always granted full Super Admin access.
- Deleting master records that are still referenced (e.g. an `InsuranceCompany` with existing claims/bills) is blocked with a friendly error; deactivation is offered instead of deletion.
- The 24-hour free-edit window on patient registration is enforced in code, with any later change routed through a fresh, separately charged Visit rather than silently overwriting history.

## 11. Technology Stack & Deployment

| Layer             | Technology                                                                                                                 |
|-------------------|----------------------------------------------------------------------------------------------------------------------------|
| Backend framework | Django 5 (Python 3)                                                                                                        |
| Database          | SQLite for development; PostgreSQL-ready for production                                                                    |
| Frontend          | Server-rendered Django templates, Bootstrap-style CSS, vanilla JS (searchable dropdowns, keyboard nav, date i              |
| Auth & roles      | Custom Django User model ( <code>AbstractUser</code> ) with a role field and per-role dashboards                           |
| Payments          | eSewa sandbox/live gateway, wired via merchant code + secret key environment variables                                     |
| QR codes          | Generated server-side (qrcode library) for patients, appointments, and printed documents                                   |
| Auditing          | Append-only <code>AuditLog</code> model, written to from every module via a shared helper                                  |
| Setup             | <code>python manage.py makemigrations / migrate / seed_nepal_address / seed_demo_data / createsuperuser / runserver</code> |

## 12. Build Status & Roadmap

### 12.1 Fully built and working (this checkpoint)

- Public hospital website, role-based authentication, Super Admin dashboard with audit trail viewer
- Departments & Doctors CRUD, staff/user account management
- Full Nepal address system (Provinces/Districts seeded, Municipality/Ward/Tole free fields)
- Patient registration with Patient ID, QR code, OPD Ticket, Patient Card, 24-hour edit window
- Online appointment booking with eSewa payment integration
- Insurance Companies module and future-ready Claims workflow
- Hospital Services catalogue
- Cash Counter / Billing with Cash / eSewa / Insurance payment, receipts, reprint auditing
- Doctor Consultation module: queue, diagnosis, prescriptions, lab & radiology requests, history
- Pharmacy + Inventory: medicine catalogue, stock adjustments, dispensing, sales, low-stock alerts
- Laboratory module: queue, status updates, result upload, printable report
- Full audit logging across every module; Django admin wired for every model

### 12.2 Noted as coming in the next checkpoint(s)

- Full Reports & Revenue Dashboard (income, department, doctor, counter, pharmacy reports)
- A dedicated, filterable Audit Trail viewer page for Super Admin (the underlying data already exists and is being written to by every module)
- Final UI polish pass across all modules

---

End of document.